

# Rust逆向之函数

## 1. 执行流程

下面的代码是函数的最基本用法：

```
fn main() {
    test_fun(3, 4);
}

fn test_fun(x1: i32, x2: i32) -> i32 {
    let mut z = 0;

    z = x1 + x2;

    z
}
```

从下面代码看出来，通过rcx和rdx来传递第一和第二个参数，然后调用函数，这和C/C++是一样的。

```
.text:0000000140001150 ; void __fastcall test_pro::main()
.text:0000000140001150 test_pro__main proc near                ; DATA XREF: main+A↓o
.text:0000000140001150                                     ;
.pdata:000000014002306C↓o
.text:0000000140001150             sub     rsp, 28h
.text:0000000140001154             mov     ecx, 3             ; int
.text:0000000140001159             mov     edx, 4             ; int
.text:000000014000115E             call    test_pro__test_fun
.text:0000000140001163             nop
.text:0000000140001164             add     rsp, 28h
.text:0000000140001168             retn
.text:0000000140001168 test_pro__main endp__main endp
```

但是进入函数以后，会先把参数保存到局部变量中，这和C/C++不一样。在C/C++中，参数是会保存到返回地址的下面，也就是参数的位置。而这里会把参数保存在返回地址上面，这个地方是在C/C++中，是用来保存局部变量的。

```

.text:0000000140001170 ; int __fastcall test_pro::test_fun(int, int)
.text:0000000140001170 test_pro__test_fun proc near
.text:0000000140001170
.text:0000000140001170             sub     rsp, 38h
.text:0000000140001174             mov     [rsp+30h], ecx ; 保存参数
.text:0000000140001178             mov     [rsp+34h], edx
.text:000000014000117C             mov     dword ptr [rsp+2Ch], 0 ; z=0
.text:0000000140001184             add     ecx, edx ; x=x+y
.text:0000000140001186             mov     [rsp+28h], ecx
.text:000000014000118A             seto    al
.text:000000014000118D             test    al, 1
.text:000000014000118F             jnz     short loc_1400011A2 ; 整型溢出则跳转
.text:0000000140001191             mov     eax, [rsp+28h] ; eax=x+y
.text:0000000140001195             mov     [rsp+2Ch], eax ; z=x+y
.text:0000000140001199             mov     eax, [rsp+2Ch] ; 返回值=x+y
.text:000000014000119D             add     rsp, 38h
.text:00000001400011A1             retn
.text:00000001400011A2 ; -----
-----
.text:00000001400011A2
.text:00000001400011A2 loc_1400011A2: ; CODE XREF:
test_pro__test_fun+1F↑j
.text:00000001400011A2             lea     rcx, aAttemptToAddWi ; "attempt to add
with overflowcalled `Opt"...
.text:00000001400011A9             lea     r8, off_140019360 ; "src\\main.rs"
.text:00000001400011B0             mov     edx, 1Ch
.text:00000001400011B5             call    _ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407
.text:00000001400011B5 ; -----
-----
.text:00000001400011BA             db 0CCh
.text:00000001400011BA test_pro__test_fun endp

```

以下是参数个数大于4个的例子:

```
fn main() {
    test_fun1(3, 4, 5, 6, 7, 8);
}

fn test_fun1(x1: i32, x2: i32, x3: i32, x4: i32, x5: i32, x6: i32) -> i32 {
    let mut z = 0;

    z = x1 + x2 + x3 + x4 + x5 + x6;

    z
}
```

当参数超过4个的时候，后面的参数会保存在栈中，这里可以看到每个参数占8字节，尽管参数类型是i32，只需要4个字节。

```
.text:0000000140001150 ; void __fastcall test_pro::main()
.text:0000000140001150 test_pro__main proc near
.text:0000000140001150
.text:0000000140001150             sub     rsp, 38h
.text:0000000140001154             mov     ecx, 3             ; int
.text:0000000140001159             mov     edx, 4             ; int
.text:000000014000115E             mov     r8d, 5             ; int
.text:0000000140001164             mov     r9d, 6             ; int
.text:000000014000116A             mov     dword ptr [rsp+20h], 7 ; int
.text:0000000140001172             mov     dword ptr [rsp+28h], 8 ; int
.text:000000014000117A             call    test_pro__test_fun1
.text:000000014000117F             nop
.text:0000000140001180             add     rsp, 38h
.text:0000000140001184             retn
.text:0000000140001184 test_pro__main endp
```

在反汇编代码中，函数会首先将这些参数都保存在局部变量中：

```

.text:00000000140001190 ; int __fastcall test_pro::test_fun1(int, int, int, int, int,
int)
.text:00000000140001190 test_pro__test_fun1 proc near
.text:00000000140001190
.text:00000000140001190             sub     rsp, 58h
.text:00000000140001194             mov     [rsp+30h], r9d ; 保存x4
.text:00000000140001199             mov     [rsp+34h], r8d ; 保存x3
.text:0000000014000119E             mov     eax, [rsp+88h] ; eax=x6
.text:000000001400011A5             mov     [rsp+38h], eax ; 保存x6
.text:000000001400011A9             mov     eax, [rsp+80h] ; eax=x5
.text:000000001400011B0             mov     [rsp+3Ch], eax ; 保存x5
.text:000000001400011B4             mov     [rsp+48h], ecx ; 保存x1
.text:000000001400011B8             mov     [rsp+4Ch], edx ; 保存x2
.text:000000001400011BC             mov     [rsp+50h], r8d ; 保存x3
.text:000000001400011C1             mov     [rsp+54h], r9d ; 保存x4
.text:000000001400011C6             mov     dword ptr [rsp+44h], 0 ; z=0

```

接下来会依次取出各个参数值进行相加，期间会判断是否发生整型溢出：

```

.text:00000001400011CE      add     ecx, edx      ; ecx=x1+x2
.text:00000001400011D0      mov     [rsp+40h], ecx ; 保存x1+x2
.text:00000001400011D4      seto    al
.text:00000001400011D7      test    al, 1
.text:00000001400011D9      jnz     short loc_1400011F2 ; 整型溢出则跳转
.text:00000001400011DB      mov     ecx, [rsp+34h] ; ecx=x3
.text:00000001400011DF      mov     eax, [rsp+40h] ; eax=x1+x2
.text:00000001400011E3      add     eax, ecx      ; eax=x1+x2+x3
.text:00000001400011E5      mov     [rsp+2Ch], eax ; 保存x1+x2+x3
.text:00000001400011E9      seto    al
.text:00000001400011EC      test    al, 1
.text:00000001400011EE      jnz     short loc_140001221 ; 整型溢出则跳转
.text:00000001400011F0      jmp     short loc_14000120A ; ecx=x4
.text:00000001400011F2 ; -----
-----
.text:00000001400011F2
.text:00000001400011F2 loc_1400011F2: ; CODE XREF:
test_pro__test_fun1+49↑j
.text:00000001400011F2      lea     rcx, aAttemptToAddWi ; "attempt to add
with overflowcalled `Opt"...
.text:00000001400011F9      lea     r8, off_140019360 ; "src\\main.rs"
.text:0000000140001200      mov     edx, 1Ch
.text:0000000140001205      call    _ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407
.text:000000014000120A ; -----
-----
.text:000000014000120A
.text:000000014000120A loc_14000120A: ; CODE XREF:
test_pro__test_fun1+60↑j
.text:000000014000120A      mov     ecx, [rsp+30h] ; ecx=x4
.text:000000014000120E      mov     eax, [rsp+2Ch] ; eax=x1+x2+x3
.text:0000000140001212      add     eax, ecx      ; eax=x1+x2+x3+x4
.text:0000000140001214      mov     [rsp+28h], eax ; 保存x1+x2+x3+x4
.text:0000000140001218      seto    al
.text:000000014000121B      test    al, 1
.text:000000014000121D      jnz     short loc_140001250 ; 整型溢出则跳转
.text:000000014000121F      jmp     short loc_140001239 ; ecx=x5
.text:0000000140001221 ; -----
-----
.text:0000000140001221
.text:0000000140001221 loc_140001221: ; CODE XREF:
test_pro__test_fun1+5E↑j
.text:0000000140001221      lea     rcx, aAttemptToAddWi ; "attempt to add
with overflowcalled `Opt"...
.text:0000000140001228      lea     r8, off_140019360 ; "src\\main.rs"
.text:000000014000122F      mov     edx, 1Ch
.text:0000000140001234      call    _ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407

```

```

.text:0000000140001239 ; -----
-----
.text:0000000140001239
.text:0000000140001239 loc_140001239: ; CODE XREF:
test_pro__test_fun1+8F↑j
.text:0000000140001239 mov ecx, [rsp+3Ch] ; ecx=x5
.text:000000014000123D mov eax, [rsp+28h] ; eax=x1+x2+x3+x4
.text:0000000140001241 add eax, ecx ; eax=x1+x2+x3+x4+x5
.text:0000000140001243 mov [rsp+24h], eax ; 保存x1+x2+x3+x4+x5
.text:0000000140001247 seto al
.text:000000014000124A test al, 1
.text:000000014000124C jnz short loc_14000127F ; 整型溢出则跳转
.text:000000014000124E jmp short loc_140001268 ; ecx=x6
.text:0000000140001250 ; -----
-----
.text:0000000140001250
.text:0000000140001250 loc_140001250: ; CODE XREF:
test_pro__test_fun1+8D↑j
.text:0000000140001250 lea rcx, aAttemptToAddWi ; "attempt to add
with overflowcalled `Opt"...
.text:0000000140001257 lea r8, off_140019360 ; "src\\main.rs"
.text:000000014000125E mov edx, 1Ch
.text:0000000140001263 call
_ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407
.text:0000000140001268 ; -----
-----
.text:0000000140001268
.text:0000000140001268 loc_140001268: ; CODE XREF:
test_pro__test_fun1+BE↑j
.text:0000000140001268 mov ecx, [rsp+38h] ; ecx=x6
.text:000000014000126C mov eax, [rsp+24h] ; eax=x1+x2+x3+x4+x5
.text:0000000140001270 add eax, ecx ; eax=x1+x2+x3+x4+x5+x6
.text:0000000140001272 mov [rsp+20h], eax ; 保存x1+x2+x3+x4+x5+x6
.text:0000000140001276 seto al
.text:0000000140001279 test al, 1
.text:000000014000127B jnz short loc_1400012A8 ; 整型溢出则跳转
.text:000000014000127D jmp short loc_140001297 ;
eax=x1+x2+x3+x4+x5+x6
.text:000000014000127F ; -----
-----
.text:000000014000127F
.text:000000014000127F loc_14000127F: ; CODE XREF:
test_pro__test_fun1+BC↑j
.text:000000014000127F lea rcx, aAttemptToAddWi ; "attempt to add
with overflowcalled `Opt"...
.text:0000000140001286 lea r8, off_140019360 ; "src\\main.rs"
.text:000000014000128D mov edx, 1Ch
.text:0000000140001292 call
_ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407

```

如果全部参数都相加完了，没有发生溢出，就会将 $x_1+x_2+x_3+x_4+x_5+x_6$ 的值保存到 $z$ 中，并将其作为返回值，然后退出函数：

```
.text:0000000140001297 ; -----  
-----  
.text:0000000140001297  
.text:0000000140001297 loc_140001297: ; CODE XREF:  
test_pro__test_fun1+ED↑j  
.text:0000000140001297 mov     eax, [rsp+20h] ; eax=x1+x2+x3+x4+x5+x6  
.text:000000014000129B mov     [rsp+44h], eax ; z=x1+x2+x3+x4+x5+x6  
.text:000000014000129F mov     eax, [rsp+44h] ; 返回值等于  
x1+x2+x3+x4+x5+x6  
.text:00000001400012A3 add     rsp, 58h  
.text:00000001400012A7 retn
```

看得出来，Rust中的函数没有像C/C++的函数一样，以返回地址作为边界，返回地址下面的地址用来保存参数，上面的地址用来保存变量。而是会将所有的参数都保存在返回地址上方，而且，将参数保存在栈上方以后，也不在像C/C++那样，每一个参数无论类型都占用8个字节。

## 2.数组和函数

以下的Rust代码，是将数组作为参数和返回值的例子：

```
fn main() {  
    let x = [1, 2, 3, 4, 5];  
  
    let z = test_array(x);  
}  
  
fn test_array(x: [i32; 5]) -> [i32; 3] {  
    let mut y = [0, 0, 0];  
  
    y[0] = x[0];  
    y[1] = x[1];  
    y[2] = x[2];  
  
    y  
}
```

main函数中的反汇编代码如下，这里可以看到。在对 $x$ 进行初始化以后，函数会将 $x$ 数组元素赋值到一块新的内存区域中，接下来，函数会将数组 $z$ 的首地址，以及刚刚复制了 $x$ 数组的内存地址分别当作第一和第二个参数来调用`test_array`函数。

```

.text:0000000140001150 ; void __fastcall test_pro::main()
.text:0000000140001150 test_pro__main proc near
.text:0000000140001150
.text:0000000140001150         sub     rsp, 58h
.text:0000000140001154         mov     dword ptr [rsp+24h], 1 ; 初始化数组x
.text:000000014000115C         mov     dword ptr [rsp+28h], 2
.text:0000000140001164         mov     dword ptr [rsp+2Ch], 3
.text:000000014000116C         mov     dword ptr [rsp+30h], 4
.text:0000000140001174         mov     dword ptr [rsp+34h], 5
.text:000000014000117C         mov     rax, [rsp+24h] ; rax=x[0]和x[1]
.text:0000000140001181         mov     [rsp+44h], rax ; 保存x[0]和x[1]
.text:0000000140001186         mov     rax, [rsp+2Ch] ; rax=x[2]和x[3]
.text:000000014000118B         mov     [rsp+4Ch], rax ; 保存x[2]和x[3]
.text:0000000140001190         mov     eax, [rsp+34h] ; rax=x[4]
.text:0000000140001194         mov     [rsp+54h], eax ; 保存x[4]
.text:0000000140001198         lea     rcx, [rsp+38h] ; rcx等于数组z的首地址
.text:000000014000119D         lea     rdx, [rsp+44h] ; rdx等于上面保存的x[0]
的首地址
.text:00000001400011A2         call    test_pro__test_array
.text:00000001400011A7         nop
.text:00000001400011A8         add     rsp, 58h
.text:00000001400011AC         retn
.text:00000001400011AC test_pro__main endp

```

在test\_array函数中，会将上面传入的复制了x数组三个元素复制给数组y。在将数组y中的元素，复制给上面传入的z数组：

```

.text:00000001400011B0 test_pro__test_array proc near
.text:00000001400011B0         sub     rsp, 10h
.text:00000001400011B4         mov     rax, rcx ; rax=rcx
.text:00000001400011B7         mov     dword ptr [rsp+4], 0 ; 初始化数组y
.text:00000001400011BF         mov     dword ptr [rsp+8], 0
.text:00000001400011C7         mov     dword ptr [rsp+0Ch], 0
.text:00000001400011CF         mov     r8d, [rdx] ; r8d=x[0]
.text:00000001400011D2         mov     [rsp+4], r8d ; y[0]=x[0]
.text:00000001400011D7         mov     r8d, [rdx+4] ; r8d=x[1]
.text:00000001400011DB         mov     [rsp+8], r8d ; y[1]=x[1]
.text:00000001400011E0         mov     edx, [rdx+8] ; r8d=x[2]
.text:00000001400011E3         mov     [rsp+0Ch], edx ; y[2]=x[2]
.text:00000001400011E7         mov     rdx, [rsp+4] ; rdx=y[0]和y[1]
.text:00000001400011EC         mov     [rcx], rdx ; z[0]=y[0], z[1]=y[1]
.text:00000001400011EF         mov     edx, [rsp+0Ch] ; edx=y[2]
.text:00000001400011F3         mov     [rcx+8], edx ; z[2]=y[2]
.text:00000001400011F6         add     rsp, 10h
.text:00000001400011FA         retn
.text:00000001400011FA test_pro__test_array endp

```

这里可以看到，数组作为参数的时候，会将数组元素复制到一块新的内存区域中，并将这块新的内存地址作为参数。而数组作为返回值的时候，main函数会拿一块内存区域作为参数去调用test\_array函数，test\_array函数就会将返回值复制到这块内存区域中，此时的rax的值也是rcx中保存的地址。而且经过调试发现，即使没有赋值，也就是上面的let z = test\_array(x);代码改成test\_array(x);，会是一模一样的汇编代码。

### 3.字符串和函数

下面的例子，是将字符串，作为函数的参数和返回值：

```
fn main() {  
    let s = "Hello";  
  
    let t = test_str(s);  
}  
  
fn test_str(s: &str) -> &str {  
    let s1 = s;  
    let s2 = "World";  
  
    s2  
}
```

从汇编代码可以看到，第一个参数和第二个参数分别是字符串的地址和字符串的长度。test\_str执行完以后，会将返回值赋给变量t：

```
.text:0000000140001150 ; void __fastcall test_pro::main()  
.text:0000000140001150 test_pro__main proc near  
.text:0000000140001150 sub rsp, 48h  
.text:0000000140001154 lea rax, aHelloworldcall ;  
"HelloWorldcalled `Option::unwrap()` on "...  
.text:000000014000115B mov [rsp+28h], rax ; 初始化变量s  
.text:0000000140001160 mov qword ptr [rsp+30h], 5  
.text:0000000140001169 lea rcx, aHelloworldcall ;  
"HelloWorldcalled `Option::unwrap()` on "...  
.text:0000000140001170 mov edx, 5  
.text:0000000140001175 call test_pro__test_str  
.text:000000014000117A mov [rsp+38h], rax ; 返回值赋给t  
.text:000000014000117F mov [rsp+40h], rdx  
.text:0000000140001184 add rsp, 48h  
.text:0000000140001188 retn  
.text:0000000140001188 test_pro__main endp
```

在test\_str函数中，会用"World"的字符串地址和长度给变量s2赋值，在通过传进来的参数s给变量s1赋值。最后，将"World"的字符串地址和长度分别赋值给eax和rdx，作为返回值：

```
.text:0000000140001190 ; ref$<str$> *__fastcall test_pro::test_str(ref$<str$> *result,
ref$<str$>)
.text:0000000140001190 test_pro__test_str proc near
.text:0000000140001190             sub     rsp, 20h
.text:0000000140001194             lea     rax, aHelloworldcall+5 ; "Worldcalled
`Option::unwrap()` on a `No"...
.text:000000014000119B             mov     [rsp], rax ; 给变量s2赋值
.text:000000014000119F             mov     qword ptr [rsp+8], 5
.text:00000001400011A8             mov     [rsp+10h], rcx ; 给变量s1赋值
.text:00000001400011AD             mov     [rsp+18h], rdx
.text:00000001400011B2             mov     edx, 5 ; 此时rax是"World"的字符
串地址，因此只需要给rdx赋值字符串的长度
.text:00000001400011B7             add     rsp, 20h
.text:00000001400011BB             retn
.text:00000001400011BB test_pro__test_str endp
```

## 4.枚举类型和函数

下面的代码是将枚举类型作为参数以及返回值的例子：

```
enum Test {
    One,
    Two,
    Three,
}

fn main() {
    let x = Test::One;
    let y = Test::Three;

    let z = test_enum(x, y);
}

fn test_enum(x: Test, y: Test) -> Test {
    let x1 = x;
    let x2 = y;

    let x3 = Test::Two;

    x3
}
```

因为枚举类型会用数字来代替，所以这里传参的时候就是把相应数字作为参数传递进去。

```
.text:0000000140001150 ; void __fastcall test_pro::main()
.text:0000000140001150 test_pro__main proc near
.text:0000000140001150
.text:0000000140001150             sub     rsp, 28h
.text:0000000140001154             mov     byte ptr [rsp+25h], 0 ; x=Test::One
.text:0000000140001159             mov     byte ptr [rsp+26h], 2 ; y=Test::Three
.text:000000014000115E             mov     cl, [rsp+25h] ; test_pro::Test
.text:0000000140001162             mov     dl, [rsp+26h] ; test_pro::Test
.text:0000000140001166             call    test_pro__test_enum
.text:000000014000116B             mov     [rsp+27h], al ; 将返回值赋给z
.text:000000014000116F             add     rsp, 28h
.text:0000000140001173             retn
.text:0000000140001173 test_pro__main endp
```

函数里面的代码和传递数字没什么区别了：

```
.text:0000000140001180 ; test_pro::Test __fastcall test_pro::test_enum(test_pro::Test,
test_pro::Test)
.text:0000000140001180 test_pro__test_enum proc near ; CODE XREF:
test_pro__main+16↑p
.text:0000000140001180 ; DATA XREF:
.pdata:0000000140023078↓o
.text:0000000140001180
.text:0000000140001180 var_3 = byte ptr -3
.text:0000000140001180 var_2 = byte ptr -2
.text:0000000140001180 var_1 = byte ptr -1
.text:0000000140001180
.text:0000000140001180             push    rax
.text:0000000140001181             mov     [rsp+6], cl ; x1=x
.text:0000000140001185             mov     [rsp+7], dl ; x2=y
.text:0000000140001189             mov     byte ptr [rsp+5], 1 ; x3=Test::Two
.text:000000014000118E             mov     al, [rsp+5] ; 返回值=x3
.text:0000000140001192             pop     rcx
.text:0000000140001193             retn
.text:0000000140001193 test_pro__test_enum endp
```

## 5.元组和函数

下面是将元组作为参数和返回值的例子：

```
fn main() {
    let x = (1, 'a');
    let t = test_tuple(x);
}

fn test_tuple(x: (i32, char)) -> (i32, char) {
    let z = (2, 'b');

    z
}
```

从下面的汇编代码可以看到，在调用test\_tuple函数的时候，会将元组中的每个元素依次作为参数传递。执行完test\_tuple函数的时候，会使用返回值来赋值变量t。

```
.text:0000000140001150 ; void __fastcall test_pro::main()
.text:0000000140001150 test_pro__main proc near
.text:0000000140001150             sub     rsp, 38h
.text:0000000140001154             mov     dword ptr [rsp+28h], 1 ; 给变量x赋值
.text:000000014000115C             mov     dword ptr [rsp+2Ch], 'a'
.text:0000000140001164             mov     ecx, [rsp+28h] ; ecx=x.0
.text:0000000140001168             mov     edx, [rsp+2Ch] ; edx=x.1
.text:000000014000116C             call    test_pro__test_tuple
.text:0000000140001171             mov     [rsp+30h], eax ; 为变量t赋值
.text:0000000140001175             mov     [rsp+34h], edx
.text:0000000140001179             add     rsp, 38h
.text:000000014000117D             retn
.text:000000014000117D test_pro__main endp
```

test\_tuple函数会将传递的参数x保存起来，然后再为变量z赋值，最后将变量z的元素赋值给eax和edx作为返回值：

```
.text:0000000140001180 ; tuple$(i32,char) __fastcall
test_pro::test_tuple(tuple$(i32,char))
.text:0000000140001180 test_pro__test_tuple proc near
.text:0000000140001180             sub     rsp, 10h
.text:0000000140001184             mov     [rsp+8], ecx ; 为变量y赋值
.text:0000000140001188             mov     [rsp+0Ch], edx
.text:000000014000118C             mov     dword ptr [rsp], 2 ; 为变量z赋值
.text:0000000140001193             mov     dword ptr [rsp+4], 'b'
.text:000000014000119B             mov     eax, [rsp] ; 将变量z的值作为返回值
.text:000000014000119E             mov     edx, [rsp+4]
.text:00000001400011A2             add     rsp, 10h
.text:00000001400011A6             retn
.text:00000001400011A6 test_pro__test_tuple endp
```

上面是元组个数比较少的情况，下面则是一个元组个数比较多的情况：

```
fn main() {
    let x = (1, 2, 3, 4, 'a', 'b');
    let t = test_tuple(x);
}

fn test_tuple(x: (i32, i32, i32, i32, char, char)) -> (i32, i32, i32, i32, char, char)
{
    let z = (5, 6, 7, 8, 'c', 'd');

    z
}
```

这个时候，对函数的调用就和将数组作为参数和返回值的时候一样。会将变量x中的值复制到一块新得内存地址中，然后将这个地址作为第二个参数，通过将变量t的地址作为第一个参数传递进去：

```
.text:0000000140001150 ; void __fastcall test_pro::main()
.text:0000000140001150 test_pro__main proc near
.text:0000000140001150
.text:0000000140001150         sub     rsp, 68h
.text:0000000140001154         mov     dword ptr [rsp+24h], 1 ; 为变量x赋值
.text:000000014000115C         mov     dword ptr [rsp+28h], 2
.text:0000000140001164         mov     dword ptr [rsp+2Ch], 3
.text:000000014000116C         mov     dword ptr [rsp+30h], 4
.text:0000000140001174         mov     dword ptr [rsp+20h], 'a'
.text:000000014000117C         mov     dword ptr [rsp+34h], 'b'
.text:0000000140001184         mov     rax, [rsp+20h] ; 将变量x中的值复制到一块
新的内存中
.text:0000000140001189         mov     [rsp+50h], rax
.text:000000014000118E         mov     rax, [rsp+28h]
.text:0000000140001193         mov     [rsp+58h], rax
.text:0000000140001198         mov     rax, [rsp+30h]
.text:000000014000119D         mov     [rsp+60h], rax
.text:00000001400011A2         lea     rcx, [rsp+38h] ; 将一块内存地址赋给rcx
.text:00000001400011A7         lea     rdx, [rsp+50h] ; 将上面复制了变量x的内存
地址赋给rdx
.text:00000001400011AC         call    test_pro__test_tuple
.text:00000001400011B1         nop
.text:00000001400011B2         add     rsp, 68h
.text:00000001400011B6         retn
.text:00000001400011B6 test_pro__main endp
```

test\_tuple函数内部也和传递数组一样，再函数内部就将数据赋值到，保存再rcx中的地址中。而此时的rax，保存的就是变量y的地址。另外，这个时候不会把传递进来的变量x保存起来。

```

.text:00000001400011C0 ; tuple$<i32,i32,i32,i32,char,char> *__fastcall
test_pro::test_tuple(tuple$<i32,i32,i32,i32,char,char> *result,
tuple$<i32,i32,i32,i32,char,char>)
.text:00000001400011C0 test_pro__test_tuple proc near          ; CODE XREF:
test_pro__main+5C↑p
.text:00000001400011C0          mov     rax, rcx          ; rax=rcx
.text:00000001400011C3          mov     dword ptr [rcx+4], 5
.text:00000001400011CA          mov     dword ptr [rcx+8], 6
.text:00000001400011D1          mov     dword ptr [rcx+0Ch], 7
.text:00000001400011D8          mov     dword ptr [rcx+10h], 8
.text:00000001400011DF          mov     dword ptr [rcx], 63h ; 'c'
.text:00000001400011E5          mov     dword ptr [rcx+14h], 64h ; 'd'
.text:00000001400011EC          retn
.text:00000001400011EC test_pro__test_tuple endp

```

## 6.结构体和函数

下面的代码是将结构体作为参数和返回值的例子：

```

struct Test {
    x: i32,
    y: char,
}

fn main() {
    let t = Test {
        x: 1,
        y: 'a',
    };

    let t1 = test_struct(t);
}

fn test_struct(t: Test) -> Test {
    let t1 = Test {
        x: 2,
        y: 'b',
    };

    t1
}

```

在传递参数的时候，和元组是一样的。一样是将结构体的成员——作为参数传入，然后获取返回值的时候，也是通过相应寄存器赋值给变量t1。

```

.text:0000000140001150 ; void __fastcall test_pro::main()
.text:0000000140001150 test_pro__main proc near
.text:0000000140001150
.text:0000000140001150         sub     rsp, 38h
.text:0000000140001154         mov     dword ptr [rsp+2Ch], 1 ; 初始化t
.text:000000014000115C         mov     dword ptr [rsp+28h], 'a'
.text:0000000140001164         mov     ecx, [rsp+28h] ; ecx=t.y
.text:0000000140001168         mov     edx, [rsp+2Ch] ; edx=t.x
.text:000000014000116C         call    test_pro__test_struct
.text:0000000140001171         mov     [rsp+30h], eax ; 将返回值赋值给t1
.text:0000000140001175         mov     [rsp+34h], edx
.text:0000000140001179         add     rsp, 38h
.text:000000014000117D         retn
.text:000000014000117D test_pro__main endp

```

test\_struct函数内部也和使用元组时候一样:

```

.text:0000000140001180 ; test_pro::Test __fastcall
test_pro::test_struct(test_pro::Test)
.text:0000000140001180 test_pro__test_struct proc near
.text:0000000140001180
.text:0000000140001180         sub     rsp, 10h
.text:0000000140001184         mov     [rsp+8], ecx ; 保存变量t
.text:0000000140001188         mov     [rsp+0Ch], edx
.text:000000014000118C         mov     dword ptr [rsp+4], 2 ; 初始化变量t1
.text:0000000140001194         mov     dword ptr [rsp], 'b'
.text:000000014000119B         mov     eax, [rsp] ; 将t1赋值给相应的寄存器
作为返回值
.text:000000014000119E         mov     edx, [rsp+4]
.text:00000001400011A2         add     rsp, 10h
.text:00000001400011A6         retn
.text:00000001400011A6 test_pro__test_struct endp

```

在看一个结构体成员比较多的情况:

```

struct Test {
    x1: i32,
    x2: i32,
    x3: i32,
    x4: i32,
    y1: char,
    y2: char,
}

fn main() {
    let t = Test {
        x1: 1,
        x2: 2,
        x3: 3,
        x4: 4,
        y1: 'a',
        y2: 'b',
    };

    let t1 = test_struct(t);
}

fn test_struct(t: Test) -> Test {
    let t1 = Test {
        x1: 5,
        x2: 6,
        x3: 7,
        x4: 8,
        y1: 'c',
        y2: 'd',
    };

    t1
}

```

从下面反汇编可以看出来，函数传参和元组个数多的时候一样：

```

.text:0000000140001150 ; void __fastcall test_pro::main()
.text:0000000140001150 test_pro__main proc near
.text:0000000140001150
.text:0000000140001150             sub     rsp, 58h
.text:0000000140001154             mov     dword ptr [rsp+30h], 1 ; 为变量t赋值
.text:000000014000115C             mov     dword ptr [rsp+34h], 2
.text:0000000140001164             mov     dword ptr [rsp+38h], 3
.text:000000014000116C             mov     dword ptr [rsp+3Ch], 4
.text:0000000140001174             mov     dword ptr [rsp+28h], 'a'
.text:000000014000117C             mov     dword ptr [rsp+2Ch], 'b'
.text:0000000140001184             lea     rcx, [rsp+40h] ; 将一块新得地址内存赋值
给rcx
.text:0000000140001189             lea     rdx, [rsp+28h] ; 将变量t的地址赋给rdx
.text:000000014000118E             call    test_pro__test_struct
.text:0000000140001193             nop
.text:0000000140001194             add     rsp, 58h
.text:0000000140001198             retn
.text:0000000140001198 test_pro__main end

```

test\_struct函数内部也和上面说的元组一样，这里就不解释了。

```

.text:00000001400011A0 ; test_pro::Test *__fastcall
test_pro::test_struct(test_pro::Test *result, test_pro::Test)
.text:00000001400011A0 test_pro__test_struct proc near ; CODE XREF:
test_pro__main+3E↑p
.text:00000001400011A0             mov     rax, rcx
.text:00000001400011A3             mov     dword ptr [rcx+8], 5
.text:00000001400011AA             mov     dword ptr [rcx+0Ch], 6
.text:00000001400011B1             mov     dword ptr [rcx+10h], 7
.text:00000001400011B8             mov     dword ptr [rcx+14h], 8
.text:00000001400011BF             mov     dword ptr [rcx], 63h ; 'c'
.text:00000001400011C5             mov     dword ptr [rcx+4], 64h ; 'd'
.text:00000001400011CC             retn
.text:00000001400011CC test_pro__test_struct endp

```